Universidad de San Carlos de Guatemala

Centro Universitario de Occidente

División de Ciencias de la Ingeniería

Sistemas Operativos 2

Laboratorio



Proyecto Final

REDXela

Jorge Anibal Bravo Rodríguez

202131782

Índice

Índice 2
1. INTRODUCCIÓN 4
2. OBJETIVOS DEL SISTEMA 4
2.1. Objetivo general 4
2.2. Objetivos especificos 4
3. ARQUITECTURA GENERAL DEL SISTEMA 5
4. TECNOLOGÍAS E INFRAESTRUCTURA 6
4.1. Backend 6
4.2. Frontend 6
4.3. Puertos y rutas base 6
5. RECURSOS Y SINCRONIZACION CON SEMAFOROS 7
5.1. Recursos modelados 7
5.2. Gestor central de recursos 7
5.3. Registro de tenedores 8
5.4. Procesamiento de un pedido 8
6. PEDIDOS, CLIENTES Y NIVELES DE SERVICIO 9
6.1. Modelo de datos 9
6.2. Registro de cuentas de clientes 10
6.3. Tipos de mercancía y recursos requeridos 10
7. COLA DE PRIORIDAD Y POLITICAS DE PLANIFICACION 11
7.1. Cola concurrente 11
7.2. Politicas de planificacion 11
7.3. Motor planificador 12
7.4. Despacho condicionado a existencias 12
7.5. Generador automático de pedidos 13
8. COMUNICACION ENTRE RECEPCIÓN E INVENTARIO 13
8.1. Buffer acotado 13
8.2. Recepción (productor) 13
8.3. Inventario (consumidor) 14
8.4. Controlador de inventario 14
9. ALMACEN CON ASIGNACIÓN NO CONTIGUA (PAGINACION) 15
9.1. Concepto aplicado 15
9.2. Estructura fisica y asignacion aleatoria 15
9.3. Tabla de paginación por lote 15
9.4. Foto del almacen para la interfaz 15
9.5. Liberacion de celdas al consumir stock 16
10. DETECCION Y RESOLUCION DE INTERBLOQUEO 16
10.1. Registro de asignación y espera 16
10.2. Detector de deadlock 16
10.3. Proceso interbloqueable y despropiacion 16
10.4. Simulador y resolución manual 17
11. API REST Y WEBSOCKET 17

12. INTERFAZ WEB..... 20
13. ESTADISTICAS DEL SISTEMA..... 22
14. EVENTOS EN TIEMPO REAL..... 22
15. DECISIONES DE DISEÑO Y FUNDAMENTO TEÓRICO..... 22
16. EJECUCION DEL SISTEMA..... 24
16.1. Requisitos..... 24
16.2. Pasos..... 24
16.3. Flujo de arranque del programa..... 25

1. INTRODUCCIÓN

El presente manual técnico describe el diseño, la arquitectura y las decisiones de implementacion del sistema de simulación del centro de distribución REDXela. Este sistema simula el funcionamiento interno de un centro de distribución de comercio electronico donde múltiples pedidos compiten por un conjunto limitado de recursos compartidos.

El sistema aplica los conceptos centrales del curso de Sistemas Operativos 2: planificacion de procesos, sincronización con semáforos, exclusión mutua con cerrojos, comunicacion entre procesos con buffers acotados, administración de almacenamiento con paginación, y detección y resolución de interbloqueos (deadlock).

2. OBJETIVOS DEL SISTEMA

2.1. Objetivo general

Simular el funcionamiento interno de un centro de distribución con múltiples pedidos concurrentes que compiten por recursos limitados, aplicando mecanismos reales de sincronización del sistema operativo.

2.2. Objetivos específicos

- Administrar seis tipos de recursos compartidos con semaforos de conteo.
- Atender pedidos según cinco niveles de servicio con politicas de planificacion intercambiables en tiempo de ejecución.
- Comunicar las areas de Recepción e Inventario con un buffer acotado sin pérdida de datos.
- Controlar el almacén físico con asignación no contigua de celdas (paginación).
- Detectar interbloqueos con un grafo de espera y resolverlos por despropiacion manual.
- Mostrar todo lo anterior en una interfaz web en tiempo real.

3. ARQUITECTURA GENERAL DEL SISTEMA

El sistema usa una arquitectura cliente servidor con dos módulos principales:

Backend: programa en Java que contiene toda la logica de simulación, la sincronización entre hilos y un servidor web HTTP con WebSocket.

Frontend: aplicación web en Svelte con Bootstrap que muestra el estado del sistema y permite interactuar con el.

La comunicación entre ambos se hace de dos formas:

API REST: el frontend pide datos con solicitudes HTTP (consultar recursos, registrar pedidos, cambiar politicas, inducir y resolver deadlock).

WebSocket: canal permanente para enviar eventos en tiempo real del backend al frontend.

Dentro del backend existen cinco grandes bloques logicos:

Gestión de recursos: semáforos que controlan el acceso a estaciones de empaque, áreas de carga, encargados de bodega, montacargas, estaciones de control de calidad y sistema de escaneo.

Planificación de pedidos: cola de prioridad con tres políticas, motor despachador con hilos y generador automático de pedidos.

Recepción e inventario: productor y consumidor conectados por un buffer acotado, mas el controlador de existencias.

Almacen paginado: matriz de celdas básicas con tablas de paginación por lote.

Detector de deadlock: registro de asignaciones, detector de ciclos y simulador de interbloqueo con resolución manual.

4. TECNOLOGÍAS E INFRAESTRUCTURA

4.1. Backend

Lenguaje: Java 25 (OpenJDK 25).

Servidor web: Javalin 6.1.3. Se eligió porque es liviano, se integra fácil con Maven y soporta rutas REST, archivos estáticos y WebSocket en pocas lineas.

Procesamiento JSON: Jackson Databind 2.17.1. Se usa para leer el cuerpo de las solicitudes POST.

Registro de eventos: SLF4J Simple 2.0.12.

Construcción: Maven. El archivo pom.xml declara las dependencias y el nivel de compilacion. El plugin shade genera un fat JAR ejecutable y el plugin antrun copia el frontend compilado dentro del JAR en la ruta public.

Sincronizacion: clases del paquete java.util.concurrent: Semaphore, ReentrantLock y Condition. No se usa sincronizacion simulada; todos los mecanismos son reales del sistema operativo a traves de la maquina virtual de Java.

4.2. Frontend

Framework: Svelte 5 con Vite 8 como herramienta de construccion.

Estilos: Bootstrap como framework principal de estilos.

Comunicación: funciones fetch para la API REST y objeto WebSocket nativo del navegador.

Estado global: almacenes (stores) de Svelte para recursos, pedidos, almacen y alertas de deadlock.

4.3. Puertos y rutas base

El servidor Javalin escucha en el puerto 7070 por defecto. Si el puerto esta ocupado, definir la variable de entorno REDXELA_PUERTO con otro valor antes de arrancar.

La interfaz web se sirve en http://localhost:7070/ (o en el puerto configurado).

La API REST vive bajo http://localhost:7070/api/.

El WebSocket vive en ws://localhost:7070/ws/simulacion.

El JAR distribuible incluye el frontend empaquetado: si la clase detecta /public/index.html en el classpath lo sirve desde ahi; si no, busca la carpeta frontend/dist relativa al directorio de trabajo; si tampoco existe, sirve solo la API REST con una advertencia en consola.

5. RECURSOS Y SINCRONIZACIÓN CON SEMÁFOROS

5.1. Recursos modelados

El archivo config/ConstantesSimulacion.java centraliza las capacidades oficiales:

6 estaciones de empaque.

2 areas de carga para camiones.

10 encargados de bodega.

3 montacargas.

4 estaciones de control de calidad.

1 sistema de escaneo compartido entre recepcion y despacho.

La enumeracion model/TipoRecurso.java representa cada tipo de recurso con su nombre y su capacidad total.

5.2. Gestor central de recursos

La clase concurrencia/GestorRecursos.java es el corazon de la sincronización. Contiene un Semaphore por cada tipo de recurso, todos creados con política de equidad (fair = true). La equidad garantiza orden FIFO entre los hilos en espera, lo cual evita inanición: ningún pedido espera para siempre mientras otros avanzan.

Operaciones principales:

adquirirRecurso(tipo, cantidad): bloquea al hilo hasta conseguir los permisos. Si el hilo es interrumpido, restaura la bandera de interrupción y retorna falso.

intentarAdquirirRecurso(tipo, cantidad, tiempo, unidad): espera como maximo un tiempo limite y retorna falso si expira.

liberarRecurso(tipo, cantidad): devuelve permisos y despierta hilos en espera.

adquirirRecursos(lista): adquiere una lista en orden; si alguna adquisición falla, libera en reversa lo ya obtenido para evitar fugas.

consultarDisponibles, consultarTotal, consultarEnUso: lectura del estado para monitoreo.

adquirirRecursoPara(tipo, idTenedor) y liberarRecursoDe(tipo, idTenedor): variantes que además anotan que pedido retiene cada recurso. Esta información alimenta al panel de recursos de la interfaz (“en uso, y por quien”).

5.3. Registro de tenedores

Además de los semáforos, el gestor mantiene un mapa de tenedores protegido con ReentrantLock: asocia cada identificador de pedido con la lista de recursos que retiene. El método obtenerTenedores(tipo) devuelve la lista de pedidos que usan un recurso en un instante dado.

Decisión de diseño: los semáforos controlan el acceso (correctitud) y el mapa de tenedores sirve solo para observación (monitoreo). Separar ambas responsabilidades evita que la interfaz interfiera con la sincronización.

5.4. Procesamiento de un pedido

La clase concurrencia/ProcesoSimulacionPedido.java implementa Runnable, es decir, cada pedido se ejecuta en su propio hilo. El flujo es:

El pedido pasa a estado EN_ESPERA y se registra el momento de inicio de espera.

Se calculan los recursos requeridos según el tipo de mercancía de sus productos.

Se adquiere cada recurso en secuencia con adquirirRecursoPara, anotando el identificador del pedido.

Al tener todos los recursos, el pedido pasa a PROCESANDO y el hilo duerme un tiempo proporcional a la cantidad de articulos (300 milisegundos por articulo, mínimo un articulo). Este tiempo simula trabajo real con los recursos retenidos.

Al despertar, el pedido pasa a COMPLETADO y se registran sus tiempos de espera y proceso en las estadísticas.

En el bloque finally se liberan los recursos en orden inverso, garantizando que nunca queden retenidos aunque el hilo sea interrumpido o cancelado.

El tiempo de procesamiento depende de la cantidad de unidades, tal como pide el enunciado: un pedido grande retiene los recursos más tiempo que uno pequeño.

6. PEDIDOS, CLIENTES Y NIVELES DE SERVICIO

6.1. Modelo de datos

model/Cliente.java: identificador, nombre y correo electrónico.

model/Producto.java: identificador, nombre, tipo de mercancía, peso y precio.

model/ItemPedido.java: producto mas cantidad solicitada.

model/Pedido.java: identificador único, cliente, nivel de servicio, lista de items, cantidad total calculada, estado del ciclo de vida, marca de tiempo de creación y conteo de reintentos (maximo 3 tras cancelaciones).

model/EstadoPedido.java: estados RECIBIDO, EN_ESPERA, PROCESANDO, COMPLETADO, CANCELADO y EN_CONFLICTO.

model/NivelServicio.java: cinco niveles con prioridad numérica y tiempo objetivo:

Expres: procesamiento casi instantáneo.

Prioritario: maximo 10 minutos en espera.

Estándar: maximo 30 minutos en espera.

Programado: maximo 60 minutos en espera.

Economico: maximo 120 minutos en espera.

6.2. Registro de cuentas de clientes

La clase service/RegistroClientes.java guarda las cuentas en un mapa protegido con ReentrantLock. Ofrece registrarCliente (rechaza nulos, vacios y duplicados), obtenerCliente, existeCliente, listarClientes y getCantidadClientes.

Al arrancar, Main registra los tres clientes del catalogo. Desde la interfaz, el panel de clientes permite crear cuentas nuevas en cualquier momento (POST /api/clientes), y el panel de pedidos exige elegir un cliente registrado (idCliente) para cada pedido manual. Si el identificador no existe, el servidor responde error 400 en vez de inventar un cliente. Asi se cumple el requisito de que un cliente nuevo pueda crear su cuenta.

6.3. Tipos de mercancía y recursos requeridos

La enumeración model/TipoMercancia.java define cinco tipos, cada uno con una combinación distinta de recursos:

Estándar: encargado de bodega y estación de empaque.

Fragil: encargado de bodega, estación de control de calidad y estación de empaque.

Pesada: encargado de bodega, montacargas y área de carga.

Valiosa: encargado de bodega, sistema de escaneo, estación de control de calidad y estación de empaque.

Refrigerada: encargado de bodega, montacargas, área de carga y sistema de escaneo.

El metodo obtenerRecursosRequeridos de Pedido recorre los items, une los recursos de cada tipo de mercancia sin duplicados y los ordena según el orden del enumerado. Este orden global es una técnica clásica de prevención del interbloqueo: si todos los hilos piden los recursos en el mismo orden, no se forma espera circular en el flujo normal.

Escenario realista de contención exigido por el enunciado: un pedido frágil y uno valioso comparten la estación de control de calidad y la estación de empaque en momentos cercanos; igualmente, un pedido de mercancía pesada y uno de refrigerada comparten montacargas y área de carga. El sistema resuelve esa contención con los semáforos: uno espera mientras el otro trabaja.

7. COLA DE PRIORIDAD Y POLITICAS DE PLANIFICACION

7.1. Cola concurrente

La clase service/ColaPrioridadPedidos.java es una cola bloqueante con prioridad dinámica. Internamente usa:

Un ReentrantLock con equidad para exclusión mutua en todas las operaciones.

Una Condition llamada noVacia para suspender hilos consumidores cuando la cola esta vacia y despertarlos al llegar un pedido.

Una lista interna que se mantiene ordenada según la política activa.

Operaciones: encolarPedido (marca el pedido en espera, inserta, reordena y señala a un consumidor), desencolarPedido (bloqueante), desencolarPedidoConTiempo (con limite de espera), cambiarPolitica (cambia el comparador y reordena todo lo en espera), obtenerPedidosEnEspera (copia defensiva para la interfaz), getCantidadEnEspera, estaVacia, getPoliticaActual, existePedidoConId (detectar duplicados) y removerPedidoPorId (extraer un pedido específico sin alterar el resto).

7.2. Politicas de planificacion

La enumeración service/PoliticaPlanificacion.java define tres políticas, cada una con su comparador:

Nivel de servicio estricto (ComparadorNivelServicio): atiende primero la prioridad mas alta (Expres antes que Económico).

Menor cantidad de artículos (ComparadorMenorCantidadArticulos): atiende primero los pedidos pequeños para liberar estaciones más rápido.

Tiempo de espera FIFO (ComparadorTiempoEsperaFifo): atiende primero al que más tiempo lleva esperando, lo cual evita inanición de pedidos de baja prioridad.

El cambio de política es dinámico: se puede hacer en cualquier momento desde la interfaz o desde código, y la cola se reordena de inmediato sin perder pedidos.

7.3. Motor planificador

La clase service/MotorPlanificador.java implementa Runnable y actúa como despachador:

Recorre la copia ordenada de la cola y remueve solo el primer pedido con existencias suficientes (seleccionarPedidoConStock con removerPedidoPorId); si ninguno califica, espera 300 milisegundos y reintenta, lo que permite detenerse con limpieza.

Crea un ProcesoSimulacionPedido, lo envuelve en un Thread y lo inicia.

Registra el hilo en una lista protegida con cerrojo y limpia periódicamente los hilos ya terminados.

Ofrece pausar, reanudar, detener, esperarHilosActivos, getCantidadHilosActivos, getEstadisticas, obtenerTareasActivas y setOyente.

Este diseño separa la decisión de orden (la cola) de la ejecución (los hilos), igual que un planificador de sistema operativo separa la cola de listos de los procesos en ejecución.

7.4. Despacho condicionado a existencias

El motor no despacha a ciegas: antes de extraer un pedido verifica con hayStockSuficiente que el inventario cubra todos sus items. Recorre la copia ordenada de la cola y remueve solo el primero que si tenga stock (removerPedidoPorId), dejando al resto en espera. Si ninguno tiene existencias, espera 300 milisegundos y reintenta. Así, cuando recepción registra mercaderia nueva, los pedidos en espera se reevaluan solos y el primero que ya pueda surtirse sale a proceso, tal como exige el enunciado.

Al terminar cada hilo, el motor hace dos tareas de cierre: si el pedido quedo COMPLETADO, descuenta sus items del inventario una sola vez (conjunto pedidosDescontados); si quedo CANCELADO, lo reencola con tope de 3 reintentos (Pedido.MAX_REINTENTOS) para no perderlo por una interrupción esporadica, y lo descarta solo si agota los intentos.

7.5. Generador automático de pedidos

La clase service/GeneradorPedidosAutomatico.java simula la llegada continua de clientes sin limite de cantidad. Cada cierto intervalo genera un pedido con cliente aleatorio del catálogo, nivel de servicio aleatorio entre los cinco, de una a tres líneas de productos y de una a cinco unidades por línea. El intervalo es configurable: 800 milisegundos por defecto, 400 milisegundos en la etapa de demostración y 2000 milisegundos en el sistema en vivo. Soporta iniciar, pausar, reanudar y detener.

8. COMUNICACIÓN ENTRE RECEPCIÓN E INVENTARIO

8.1. Buffer acotado

La clase service/BufferRecepcion.java implementa el patrón productor consumidor con tres primitivas:

semaforoEspacios: cuenta los lugares libres. El productor lo decrementa antes de insertar; si vale cero, el productor se bloquea (no se pierde mercaderia aunque recepcion sea mas rapida).

semaforoElementos: cuenta los lotes listos. El consumidor lo decrementa antes de extraer; si vale cero, el consumidor se bloquea (no consume basura cuando no hay datos).

cerrojoMutex: ReentrantLock que protege la lista interna para que solo un hilo a la vez inserte o extraiga.

Ambos semaforos se crean con equidad para un trato justo entre hilos. Tambien existen variantes con tiempo limite (intentarEnviarLote, recibirLoteConTiempo) para operaciones que no deben bloquear para siempre.

8.2. Recepción (**productor**)

La clase service/ServicioRecepcion.java representa el área de recepción. Puede operar de tres formas:

Manual: recibirLoteDeCamion deposita un lote específico en el buffer, e ingresarLoteManual valida producto y cantidad contra el catálogo para el registro desde la interfaz.

Aleatoria: generarLoteAleatorio crea lotes de 2 a 6 unidades de un producto del catálogo.

Continúa: su método run genera lotes cada cierto intervalo en segundo plano; en el sistema en vivo el hilo Hilo-Recepcion-Viva genera un lote cada 2.5 segundos para alimentar stock de forma permanente.

Desde la interfaz no se crean productos nuevos: se selecciona uno del catálogo y se indica la cantidad, tal como pide el enunciado.

8.3. Inventario (**consumidor**)

La clase service/ServicioInventario.java representa el área de control de inventario. Extrae lotes del buffer con tiempo límite, simula el tiempo de clasificación y estibado, y delega el registro al ControladorInventario. Puede procesar un lote puntual (procesarSiguienteLote) o correr en segundo plano (run).

8.4. Controlador de inventario

La clase service/ControladorInventario.java mantiene las existencias por producto en un mapa protegido con cerrojo. El metodo procesarEntradaLote asigna celdas en el almacen paginado, luego incrementa el stock y registra el lote para consumo posterior; si no hay espacio físico, retorna falso y no toca el stock, manteniendo la consistencia. Tambien ofrece descontarStock, consultarStock y tieneStockSuficiente para atender la demanda de los pedidos.

Al descontar, consume los lotes más antiguos primero (FIFO) y libera con liberarEspacioLote las celdas de los lotes agotados. La liberación se hace fuera del

cerrojo de stock para no invertir el orden de bloqueo con el almacen, evitando un interbloqueo entre el hilo de inventario y el hilo del motor.

9. ALMACEN CON ASIGNACIÓN NO CONTIGUA (PAGINACIÓN)

9.1. Concepto aplicado

El almacén se modela como memoria paginada: un lote es como un proceso que necesita cierto número de páginas (celdas), y esas celdas pueden quedar dispersas en cualquier pasillo y nivel según el espacio libre del momento. Así se cumple el requisito de que un lote puede quedar repartido sin necesidad de ubicaciones contiguas.

9.2. Estructura física y asignación aleatoria

La clase almacen/AlmacenPaginado.java crea una matriz tridimensional de celdas: pasillos por niveles por espacios por nivel. La configuracion usada en la simulación es de 4 pasillos, 3 niveles y 6 espacios por nivel, es decir, 72 celdas en total. Toda asignación y liberación se hace bajo un ReentrantLock con equidad.

Para asignar un lote, el método asignarEspacioNoContiguo recolecta todas las celdas libres, las mezcla al azar y ocupa las primeras de la lista mezclada. Asi cada lote queda repartido en pasillos y niveles distintos en vez de llenar siempre desde el pasillo 1, que es lo que exige el enunciado sobre ubicaciones no contiguas.

9.3. Tabla de paginación por lote

Cada lote registrado obtiene una almacen/TablaPaginacionLote.java que asocia su identificador con la lista de almacen/UbicacionAlmacen.java asignadas (cada ubicación guarda pasillo, nivel, espacio, lote y producto). El método consultarTablaLote permite ver exactamente donde quedó cada unidad, y liberarEspacioLote desocupa todas las celdas de un lote de una sola vez.

9.4. Foto del almacen para la interfaz

El método obtenerEstadoCeldas devuelve una copia del estado de las 72 celdas (pasillo, nivel, espacio, ocupada y lote asignado) tomada bajo cerrojo. El endpoint GET /api/almacen serializa esa foto celda por celda, de modo que el mapa de la interfaz colorea en verde cada celda ocupada con su lote y en gris cada celda libre, con resumen de ocupadas, libres y capacidad total.

9.5. Liberación de celdas al consumir stock

El ControladorInventario lleva por producto la lista de lotes en orden de llegada y las unidades pendientes de cada uno. Al descontar stock, consume los lotes más antiguos primero (FIFO) y libera con liberarEspacioLote las celdas de los lotes agotados. La liberación se hace fuera del cerrojo de stock para no invertir el orden de bloqueo con el almacen. Asi el mapa respira: las celdas se ocupan con la llegada de mercadería y se liberan a medida que los pedidos consumen existencias, sin fugas de espacio.

10. DETECCION Y RESOLUCION DE INTERBLOQUEO

10.1. Registro de asignación y espera

La clase concurrencia/deadlock/RegistroAsignacionRecursos.java mantiene el grafo de asignación con dos mapas protegidos por cerrojo:

Recursos retenidos por cada pedido (aristas de asignación).

Recurso que cada pedido está esperando (aristas de solicitud).

Es la estructura equivalente a la matriz de asignación y el vector de espera de los algoritmos clásicos de detección.

10.2. Detector de deadlock

La clase concurrencia/deadlock/DetectorDeadlock.java busca ciclos de longitud dos: para cada par de pedidos en espera verifica si el primero retiene lo que el segundo espera y viceversa. Si ambas condiciones se cumplen, construye un

InformacionConflicto con los identificadores, los recursos retenidos y los solicitados, más un texto explicativo de la causa del bloqueo.

10.3. Proceso interbloqueable y despropiacion

La clase concurrencia/deadlock/ProcesoPedidoInterbloqueable.java es una tarea diseñada para provocar el interbloqueo de forma controlada: adquiere su primer recurso, hace una pausa para que el otro hilo adquiera el suyo, registra la solicitud del segundo recurso y luego intenta adquirirlo quedando bloqueada. El método desapropiar marca la bandera volátil, interrumpe el hilo ejecutor y, en el bloque finally, libera el recurso retenido y devuelve el pedido a estado EN_ESPERA para que vuelva a la lista de espera.

10.4. Simulador y resolución manual

La clase concurrencia/deadlock/SimuladorDeadlock.java coordina el escenario:

forzarDeadlockDeliberado satura los recursos sobrantes para dejar exactamente una unidad libre de cada recurso disputado, lanza dos tareas en orden inverso, espera a que ambas se bloqueen y ejecuta el detector.

resolverConflictoManual(idGanador) despropia al perdedor, espera que el ganador termine, restaura los permisos retenidos y marca el conflicto como resuelto.

getPedidoPerdedor permite reencolar al perdedor para que sea atendido mas adelante.

En la interfaz, el panel de deadlock muestra los dos pedidos, que recurso retiene cada uno y cual espera, y ofrece botones para inducir el conflicto y resolverlo eligiendo al ganador.

11. API REST Y WEBSOCKET

El servidor expone los siguientes endpoints, todos implementados en controller/ControladorServidor.java:

Método	**Ruta**	**Propósito**
GET	/api/estado	Resumen general del sistema en JSON
GET	/api/recursos	Disponibles, totales, en uso y tenedores por recurso
GET	/api/procesando	Pedidos en ejecucion con progreso y tiempo restante
GET	/api/cola	Politica activa, cantidad en espera y lista ordenada
GET	/api/estadisticas	Completados y promedios de espera y proceso
GET	/api/almacen	Capacidad, ocupacion y celdas del almacen
GET	/api/productos	Catalogo de productos disponibles
GET	/api/deadlock/detalle	Estado del ultimo conflicto detectado
GET	/api/clientes	Listado de cuentas de clientes registrados

GET	/api/bitacora	Ultimos 200 eventos de la simulacion para la bitacora visible
POST	/api/pedidos	Registro manual de un pedido (idCliente, nivel, items con producto y cantidad)
POST	/api/clientes	Registro manual de una cuenta de cliente (id, nombre, correo)
POST	/api/politica	Cambio dinamico de politica de planificacion
POST	/api/recepcion	Registro manual de mercaderia (producto, cantidad)
POST	/api/deadlock/inducir	Provoca un interbloqueo deliberado (respuesta en texto plano)
POST	/api/deadlock/resolver	Resuelve el conflicto indicando el ganador
WS	/ws/simulacion	Canal de eventos en tiempo real

Las respuestas de consulta y la mayoría de POST son objetos JSON construidos en el servidor (inducir deadlock responde texto plano). Las solicitudes POST reciben cuerpos JSON que se leen con Jackson. Los errores de formato devuelven codigo 400 con un mensaje descriptivo.

12. INTERFAZ WEB

La interfaz está en frontend/src y se compila con Vite a la carpeta frontend/dist, la cual sirve el propio Javalin (empaquetada en el JAR bajo la ruta public, o como carpeta externa en desarrollo). Sus piezas son:

App.svelte: tablero principal con diez paneles en cuadricula Bootstrap. Al iniciar pide el estado al backend y luego actualiza cada 2 segundos por polling; además mantiene un WebSocket para eventos en tiempo real, y cada mensaje dispara un refresco inmediato.

PanelRecursos: tabla por recurso con disponibles, totales, en uso, barra de ocupación y lista de tenedores.

PanelCola: política activa, cantidad en espera y lista ordenada con posición, cliente, nivel, articulos y espera acumulada.

PanelProcesando: pedidos en ejecución con barra de progreso, tiempo restante y tiempo de espera.

PanelEstadisticas: tarjetas con completados, promedios de espera y proceso, en espera y en proceso.

PanelPedidos: formulario de registro manual: selector de cliente registrado, selector de nivel, constructor de líneas (producto del catálogo mas cantidad, con botón de agregar y quitar) y selector de política. El servidor asigna el identificador (WEB-N correlativo) y valida cliente, productos y cantidades.

PanelClientes: formulario de creación de cuentas (id, nombre, correo) más lista de cuentas registradas.

PanelMercaderia: formulario de registro manual de mercadería con catálogo de productos.

PanelDeadlock: estado del conflicto, pedidos involucrados con recursos obtenidos y esperados, botones de inducir y resolver.

MapaAlmacen: consulta GET /api/almacen cada 2 segundos y dibuja la cuadrícula por pasillo y nivel: celdas ocupadas en verde con su lote y libres en gris, con leyenda y resumen de ocupación.

PanelBitacora: muestra los últimos 200 eventos de la simulación (GET /api/bitacora) en consola oscura con desplazamiento automático y casilla para pausar el seguimiento. Refleja en la interfaz las mismas etapas que se imprimen en la terminal del servidor.

servicioApi.js: funciones de acceso a cada endpoint REST (recursos, procesando, cola, estadísticas, almacen, productos, clientes, pedidos, politica, recepción, deadlock, bitacora).

tiendaSimulacion.js: almacenes globales de Svelte para compartir datos entre paneles.

13. ESTADÍSTICAS DEL SISTEMA

La clase service/RegistroEstadisticas.java acumula métricas con exclusión mutua: total de pedidos completados, suma de tiempos de espera y suma de tiempos de proceso. Expone los promedios calculados al momento de la consulta. Cada ProcesoSimulacionPedido registra sus tiempos al completarse, de modo que las estadísticas reflejan el comportamiento real de la simulación y permiten comparar políticas entre sí.

14. EVENTOS EN TIEMPO REAL

La interfaz service/OyenteSimulacion.java desacopla el motor del servidor web: el motor solo avisa (alDespacharPedido, alCompletarPedido) y el ControladorServidor implementa el aviso difundiendo un mensaje JSON a todas las sesiones WebSocket conectadas (difundirEvento). La inducción y la resolución de deadlock también difunden alCambiarDeadlock. El frontend escucha el canal y refresca el tablero de inmediato, sin esperar el siguiente ciclo de polling de 2 segundos. Las sesiones se registran al conectar y se remueven al desconectar en un conjunto concurrente, por lo que varios navegadores pueden observar a la vez. Además del canal push, la

bitacora de eventos (util/RegistroEventos, alimentada por util/ConsolaFormato en cada mensaje de consola) se consulta por polling en GET /api/bitacora, de modo que ningún evento de las etapas se pierde aunque el WebSocket se caiga.

15. DECISIONES DE DISEÑO Y FUNDAMENTO TEÓRICO

Semáforos de conteo con equidad: cada recurso se modela con un Semaphore inicializado en su capacidad. La equidad evita inanición entre hilos competidores.

Exclusión mutua con ReentrantLock: todas las estructuras compartidas (cola, buffer, almacen, registro, estadísticas) se protegen con cerrojos con equidad en lugar de synchronized, porque permiten equidad, condiciones de espera y limpieza garantizada con try finally.

Productor consumidor con doble semáforo: el buffer usa un semáforo de espacios y uno de elementos más un mutex, que es la solución canónica del problema y evita pérdida de datos y bloqueos indefinidos.

Orden global de recursos: los pedidos normales adquieren recursos en orden fijo del enumerado, lo cual rompe la condicion de espera circular de Coffman en el flujo normal; el deadlock solo aparece en el escenario deliberado que usa orden inverso a proposito.

Cambio dinamico de politica: la cola separa el criterio de orden (comparador) de los datos (lista), por eso cambiar de politica es solo cambiar el comparador y reordenar.

Tiempo proporcional a unidades: el trabajo simulado dura 300 milisegundos por articulo, cumpliendo que un pedido grande consuma mas recursos que uno pequeno.

Despropiacion como resolucion: en lugar de matar procesos, se interrumpe al perdedor, se le quita el recurso y se le devuelve a la cola, que es una de las tres estrategias clasicas de recuperacion de deadlock.

Monitoreo sin interferencia: los tenedores, el progreso y los tiempos se calculan con lecturas protegidas que nunca bloquean a los hilos de trabajo mas de lo necesario.

Servidor primero con instancias compartidas: el servidor arranca antes de las etapas de demostracion y todo (cola, gestor, almacen, inventario) usa las mismas instancias, asi la interfaz observa el sistema real desde el primer segundo y no existen copias desconectadas.

Empaquetado portable: el frontend viaja dentro del JAR bajo /public y el servidor prefiere el classpath, con respaldo a carpeta externa y modo solo-API si no hay interfaz; el puerto se configura con REDXELA_PUERTO.

Asignacion aleatoria y liberacion FIFO: el almacen mezcla las celdas libres para repartir cada lote y libera las celdas de los lotes agotados al consumir stock, fuera del cerrojo de stock para no invertir el orden de bloqueo.

Reintentos con tope: los pedidos cancelados vuelven a la cola hasta 3 veces (Pedido.MAX_REINTENTOS) en vez de perderse o rebotar para siempre.

Notificacion doble: eventos push por WebSocket mas bitacora por polling, de modo que la interfaz en tiempo real no depende de un solo canal.

16. EJECUCION DEL SISTEMA

16.1. Requisitos

JDK 25 instalado.

Maven instalado.

Node.js y npm para compilar el frontend (solo la primera vez o cuando cambie la interfaz).

16.2. Pasos

Compilar el frontend: entrar a frontend, ejecutar npm install y luego npm run build. Esto genera la carpeta frontend/dist.

Compilar el backend: entrar a backend y ejecutar mvn clean compile.

Empaquetar el distribuible: ejecutar mvn package para generar target/backend-1.0-SNAPSHOT.jar (fat JAR con dependencias y frontend incluido en

la ruta public; existe una copia lista en la raiz del repositorio). El plugin antrun copia frontend/dist al JAR automaticamente; si la carpeta no existe, el JAR sirve solo la API.

Ejecutar: java -jar backend-1.0-SNAPSHOT.jar desde cualquier carpeta, o mvn exec:java, o correr la clase com.redxela.Main desde el IDE. El servidor queda escuchando en el puerto 7070 (o el indicado en REDXELA_PUERTO) con el motor y el generador activos.

Abrir el navegador en http://localhost:7070/ para usar la interfaz.

16.3. Flujo de arranque del programa

La clase Main ejecuta en orden: registro de clientes del catalogo, prueba rapida de cola y politicas, creacion de los servicios compartidos (cola, gestor, almacen e inventario), arranque inmediato del servidor Javalin para que la interfaz observe todo lo que sigue, comunicacion recepcion inventario sobre las instancias vivas (el almacen y el stock quedan visibles en la interfaz), asignacion paginada en almacen, planificacion concurrente con generador automatico, verificacion de integridad de semaforos, demostracion de deadlock con resolucion manual y, al final, arranque del motor y el generador en vivo con validacion de stock (incluido el hilo de recepcion continua).